

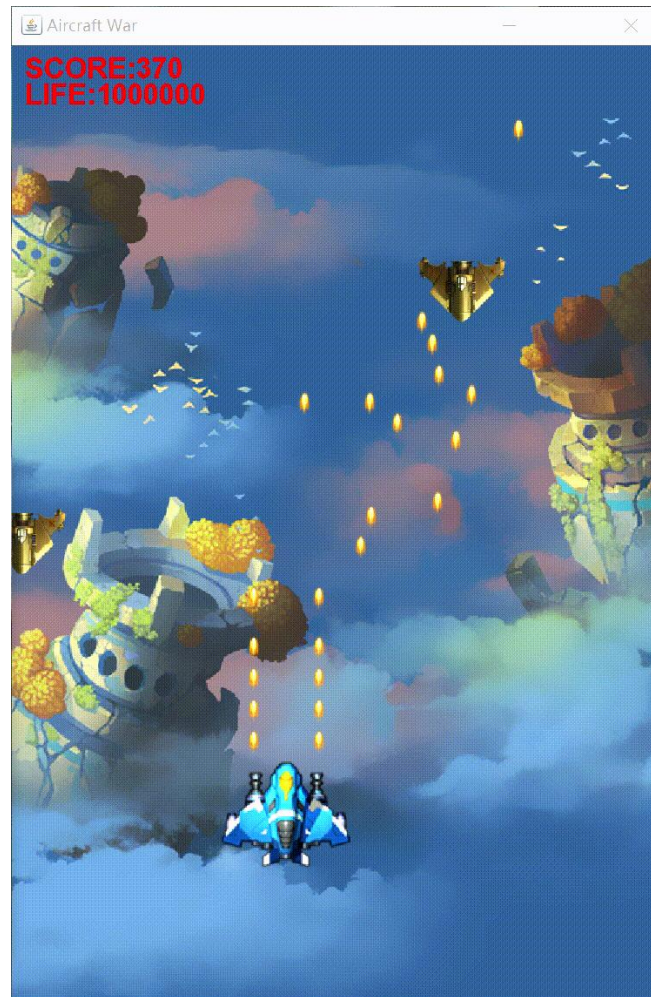
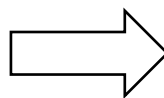
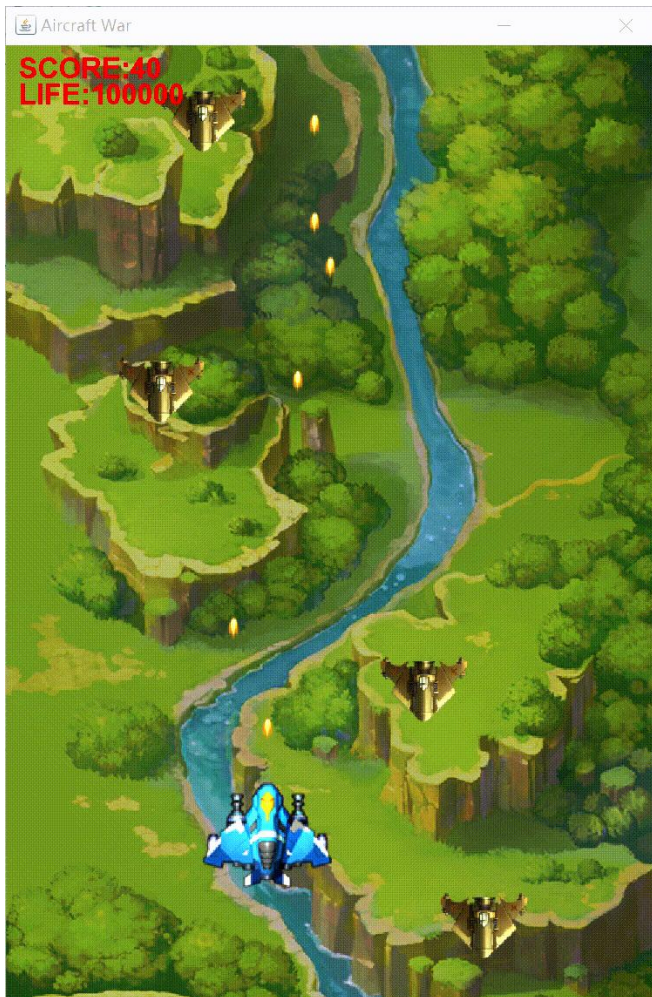


实验一：系统分析及单例模式

实验与创新实践教育中心 • 计算机与数据技术实验教学部

本学期实验总体安排

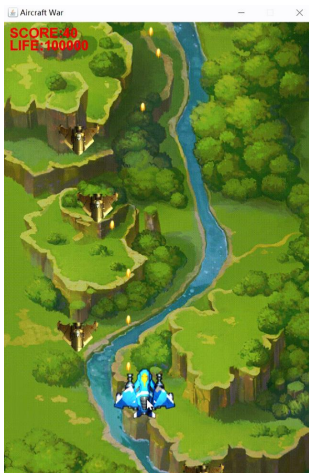
我们的
开始



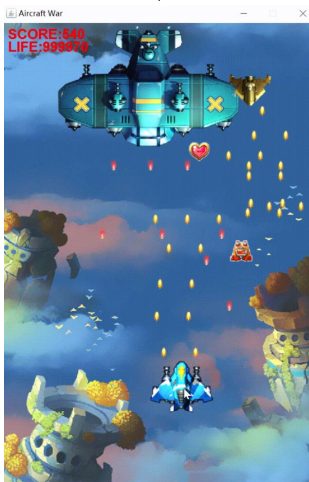
我们的
目标 (更多惊喜)

本学期实验总体安排

初始版本



最终版本



游戏主界面
英雄机移动
英雄机子弹直射
碰撞检测
统计得分和生命值

采用**简单工厂模式**
创建道具
采用**工厂模式**
创建敌机

采用**数据访问对象模式**
实现得分排行榜
添加**JUnit**单元测试

采用**观察者模式**
实现炸弹、冰冻道具
采用**模板模式**
实现三种游戏难度

初始版本

01

UML建模

精英敌机生成、攻击
道具生成、加血道具生效
采用**单例模式**创建英雄机



02

03

采用**策略模式**

实现不同飞机的发射弹道
两种火力道具生效

04

05

使用**Swing**添加游戏难度选择和
排行榜界面
使用**多线程**实现音效控制及火力
道具状态恢复

06

本学期实验总体安排

实验项目	一	二	三	四	五	六
学时数	2	2	2	2	4	4
实验内容	系统分析 单例模式	简单工厂 工厂模式	策略模式	JUnit 数据访问 对象模式	Swing 多线程	观察者模式 模板模式
时间节点			中期			结项
提交内容			UML类图 项目代码			结项报告 代码、类图 展示视频

实验课程共 **16** 个学时，**6** 个实验项目，总成绩为 **30分**。

CONTENTS

目录

01 实验目的

02 实验任务

03 实验步骤

04 迭代目标

实验目的

- 理解**面向对象**编程的基本思想和核心概念；
- 结合具体实例，掌握面向对象**分析与设计**的方法；
- 掌握**UML类图**绘制规范，能够使用类图准确表达系统的继承关系；
- 深入理解**单例模式**的设计意图及UML结构，并能通过代码实现与应用。



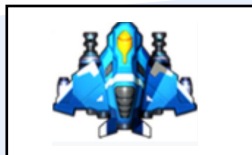
实验任务

1. 理解飞机大战系统功能，导入并运行模板程序；
2. 分析核心实体类的继承关系并完成UML建模，用PlantUML绘制完整类图
3. 重构代码，新增 4 种敌机、5 种道具及其抽象父类，完善继承体系
4. 理解单例模式设计意图，绘制英雄机单例模式的UML结构图
5. 重构英雄机创建逻辑，采用单例模式保证全局唯一实例
6. 完成本次迭代开发清单中的所有任务

实验步骤：飞机大战任务书（1）

角色设定

- 英雄机



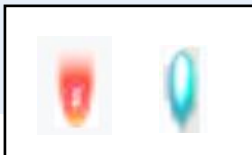
- 敌机



- 道具



- 子弹



实验步骤：飞机大战任务书（2）

游戏规则

- **游戏难度**：简单、普通、困难
- 子弹发射、弹道变化
- 道具生成、使用 and 效果
- 生命值和得分计算、总分排行榜

实验步骤：飞机大战任务书（3）

界面和
音效

- 游戏地图
- 音效的开启和关闭

需求细则请参考实验指导书！

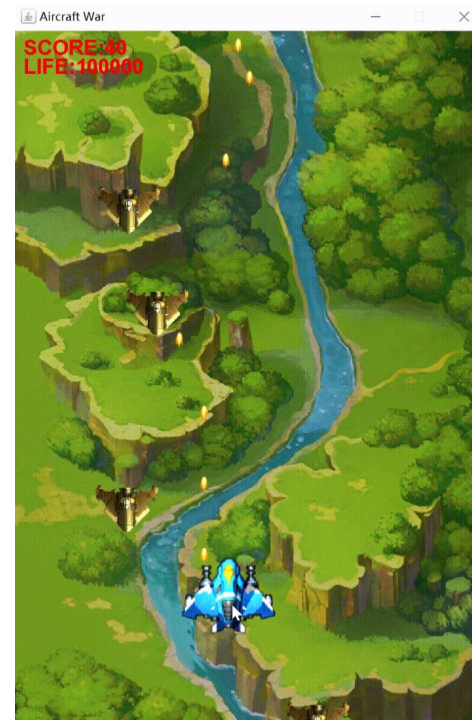
实验步骤：模板程序导入及运行（1）

- 下载并安装java集成开发环境 IntelliJ IDEA（社区版 2025.3）

<https://www.jetbrains.com/zh-cn/idea/download/#section=windows>

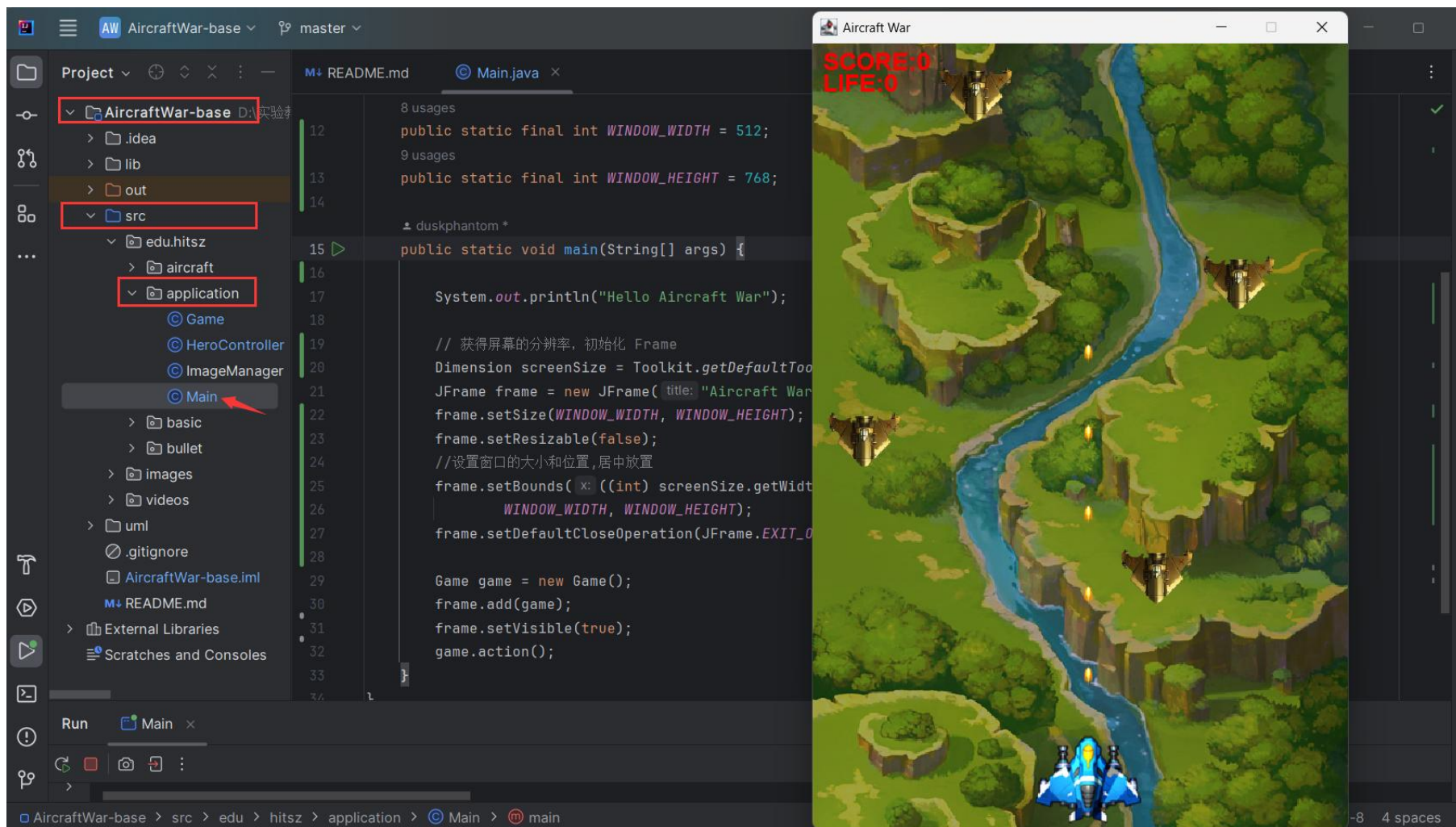
- 本实验提供了两个版本的模板代码 **AircraftWar-base**，已实现如下功能：

- 游戏主界面
- 英雄机、普通敌机移动
- 英雄机子弹直射
- 碰撞检测
- 统计并显示得分和英雄机生命值



实验步骤：模板程序导入及运行（2）

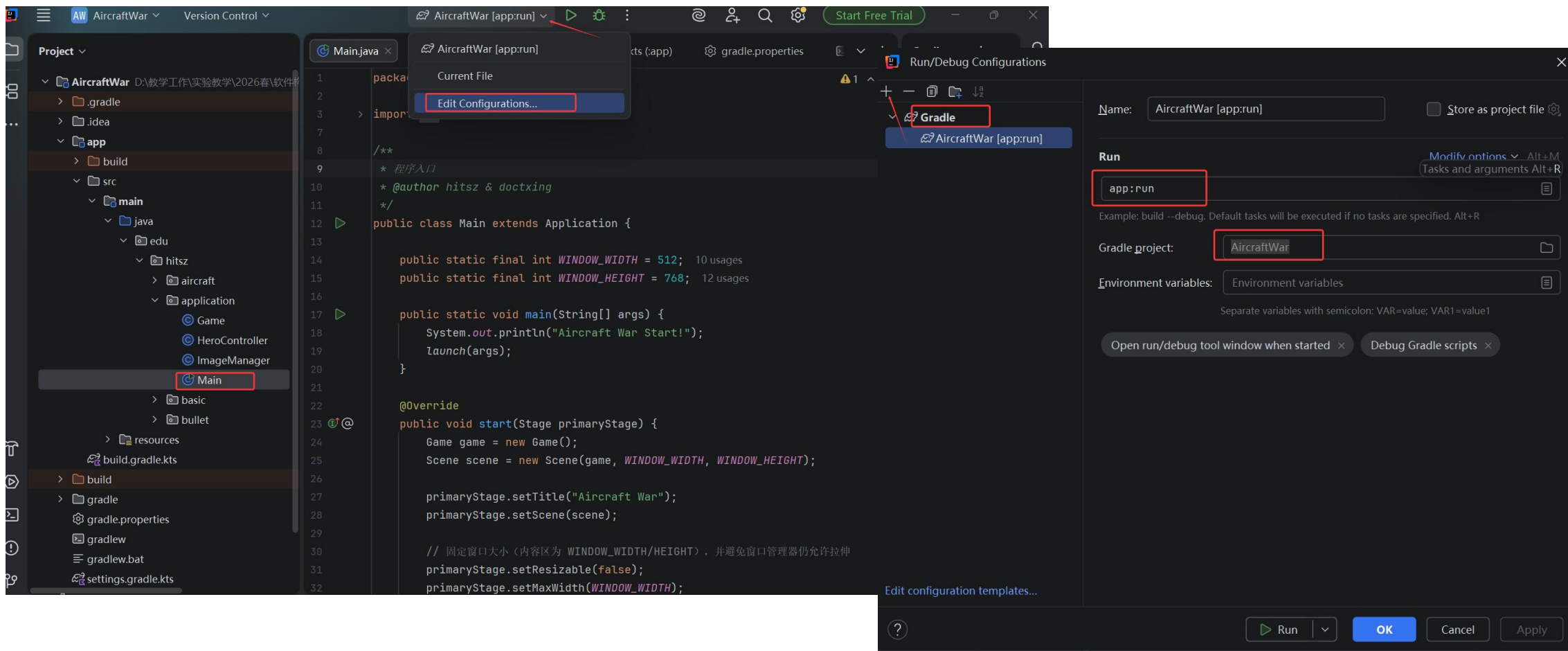
➤ **Java Swing版本：** 可运行src\edu\hitsz\application\Main.java 启动程序



实验步骤：模板程序导入及运行（2）

➤ **JavaFX版本**：通过 Gradle 运行项目（确保 Gradle 已完成项目同步）

Edit Configurations -> 点击 + 号，选择 Gradle -> 右侧面板设置 -> 点击  运行



The screenshot illustrates the process of running a JavaFX project in IntelliJ IDEA. The main editor shows the `Main.java` file with the following code:

```
1 package ...
2 import ...
3
4 /**
5  * 程序入口
6  * @author hitsz & doctxing
7  */
8
9 public class Main extends Application {
10
11     public static final int WINDOW_WIDTH = 512; 10 usages
12     public static final int WINDOW_HEIGHT = 768; 12 usages
13
14     public static void main(String[] args) {
15         System.out.println("Aircraft War Start!");
16         launch(args);
17     }
18
19     @Override
20     public void start(Stage primaryStage) {
21         Game game = new Game();
22         Scene scene = new Scene(game, WINDOW_WIDTH, WINDOW_HEIGHT);
23
24         primaryStage.setTitle("Aircraft War");
25         primaryStage.setScene(scene);
26
27         // 固定窗口大小（内容区为 WINDOW_WIDTH/HEIGHT），并避免窗口管理器仍允许拉伸
28         primaryStage.setResizable(false);
29         primaryStage.setMaxWidth(WINDOW_WIDTH);
30     }
31 }
```

The `Main` class is highlighted in the Project view. The `Edit Configurations` dialog is open, showing the `Gradle` configuration for the `AircraftWar [app:run]` task. The `Run` tab is selected, and the `app:run` task is chosen. The `Gradle project` is set to `AircraftWar`. The `Environment variables` section is also visible.

实验步骤：核心实体类设计与UML建模（1）

核心实体类：英雄机（1种）、敌机类（5种）、道具类（5种）、子弹类（2种）

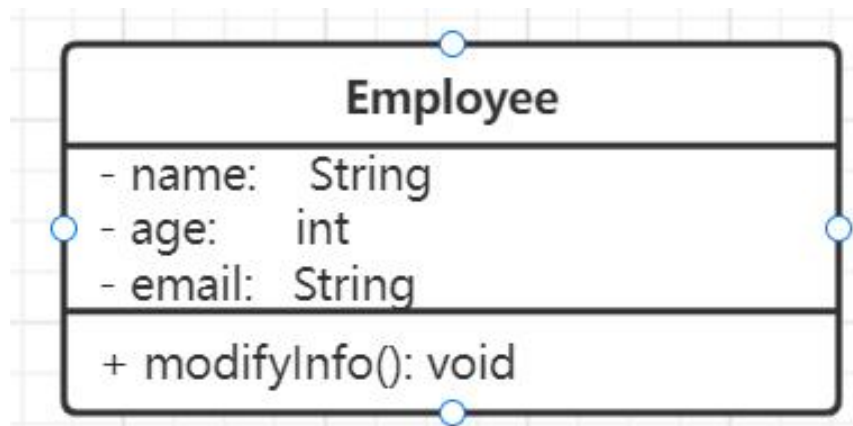
请依据**面向对象设计原则**（如封装、继承、多态等），**分析并设计**上述所有类的层次结构。需合理提取公共特征以构建抽象父类，并派生出对应的具体子类。

基于上述设计，编辑项目**uml/Inheritance.puml** 文件（使用 PlantUML 插件），绘制完整 UML 类图。

要求：完整展示所有具体类、抽象父类及继承关系，清晰体现系统层次结构。

实验步骤：核心实体类设计与UML建模（2）

类图 (Class Diagram)：用来显示系统中的类、接口以及它们之间的静态结构和关系的一种静态模型。



```
public class Employee {  
  
    private String name;  
    private int age;  
    private String email;  
  
    public void modifyInfo() {  
        .....  
    }  
}
```


实验步骤：核心实体类设计与UML建模（3）

类与类之间的关系： 关联、泛化、实现和依赖。

其中**关联**又分为**一般关联**、**聚合关系**和**组合关系**。

01 ↑

泛化关系

继承关系，子类继承父类的所有行为和属性。如：老虎和动物。

02 ↑

实现关系

类与接口的关系，表示类是接口所有特征和行为的实现者。如：鸟和飞行。

03 ↓

依赖关系

一种**使用关系**，一个类的实现需要其他类的协助。如：驾驶员和汽车。

04 ↓

一般关联

对象之间的一种**引用关系**，用于表示一类对象与另一类对象之间的联系。如：老师和学生。

05 ↓

聚合关系

整体与部分的关系，且部分可以离开整体而单独存在。如：汽车和轮胎。

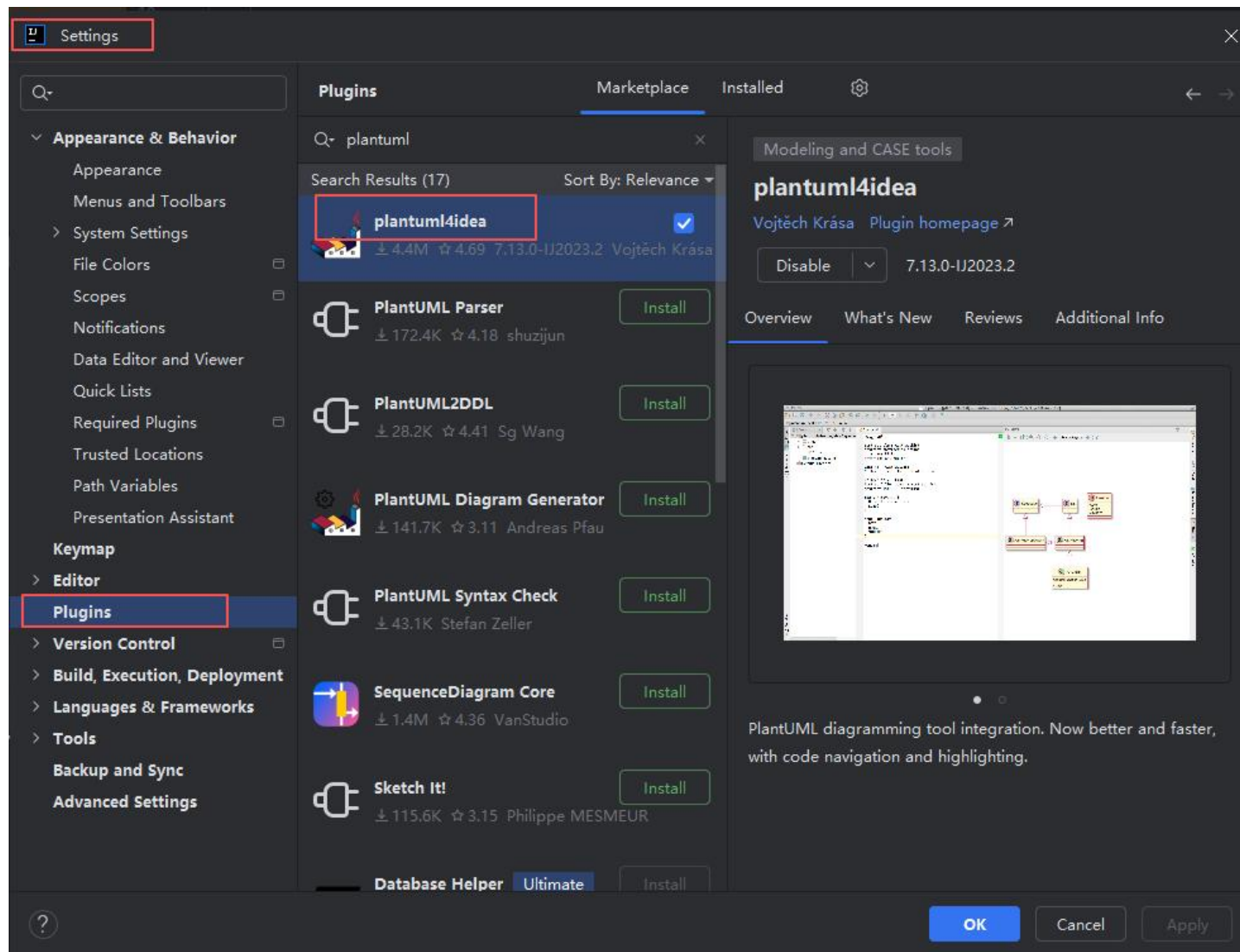
06 ↓

组合关系

整体与部分的关系，但部分不能离开整体而单独存在。如：公司和部门。

实验步骤：PlantUML插件 (1)

➤ **PlantUML**是一个开源项目，支持快速绘制**类图**、时序图、用例图等UML结构图和行为图。



实验步骤：PlantUML插件（2）

➤ **PlantUML** 类图及相关语法请参考官网说明：

<https://plantuml.com/class-diagram>

➤ 接口、抽象类、类的画法：

```
@startuml
'https://plantuml.com/class-diagram

interface 接口
{
}

abstract class 抽象类
{
}

class 类
{
    + 公有成员变量: String
    - 私有成员变量: int
    # 受保护的成员变量: Date
    + {static} 静态成员变量: String

    + 构造函数(int 参数1)
    + 普通方法(int 参数1, String 参数2): void
    + {abstract} 抽象方法(): int
    + {static} 静态方法(int 参数1, String 参数2): String
}
```



接口



抽象类

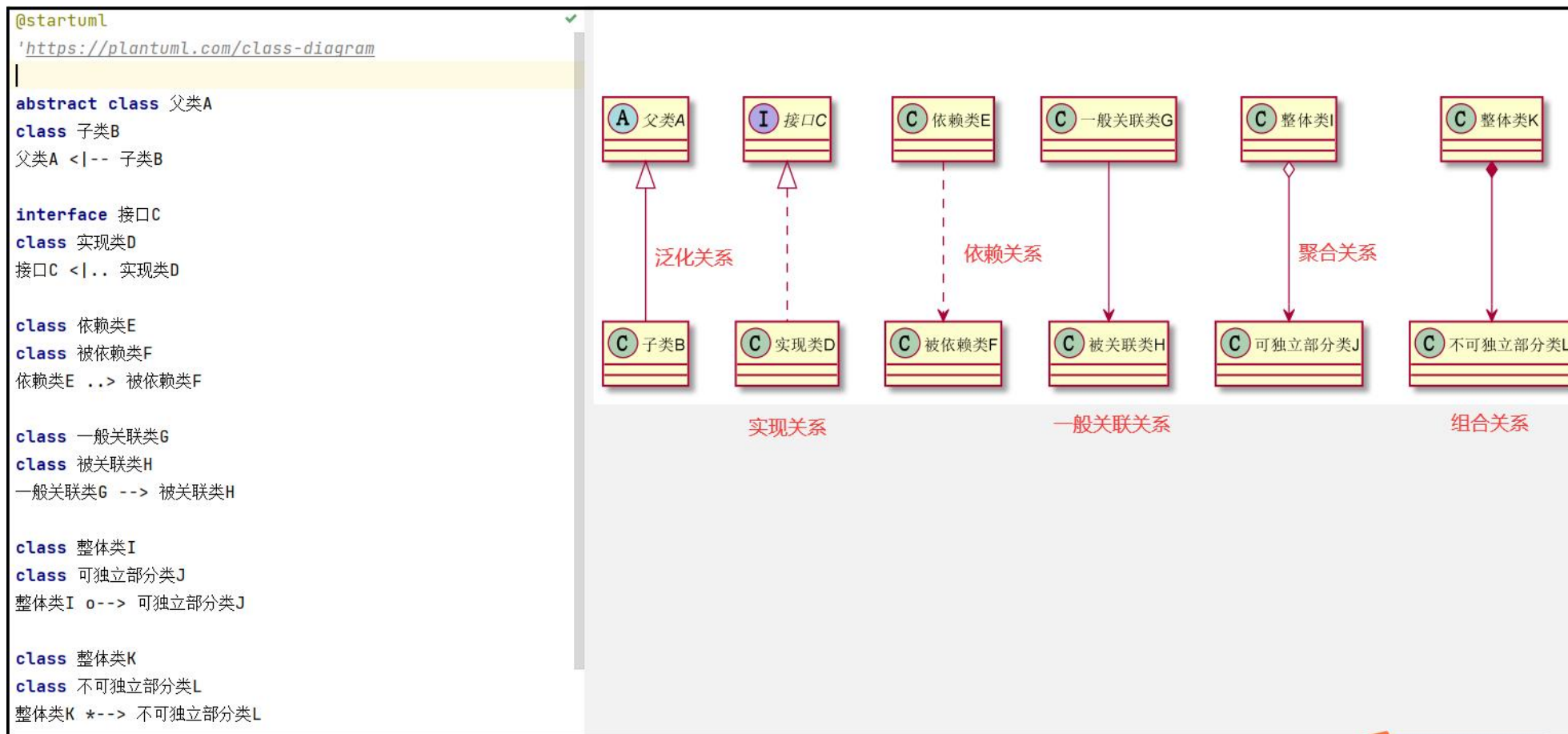


类

- 公有成员变量: String
- 私有成员变量: int
- ◇ 受保护的成员变量: Date
- 静态成员变量: String
- 构造函数(int 参数1)
- 普通方法(int 参数1, String 参数2): void
- 抽象方法(): int
- 静态方法(int 参数1, String 参数2): String

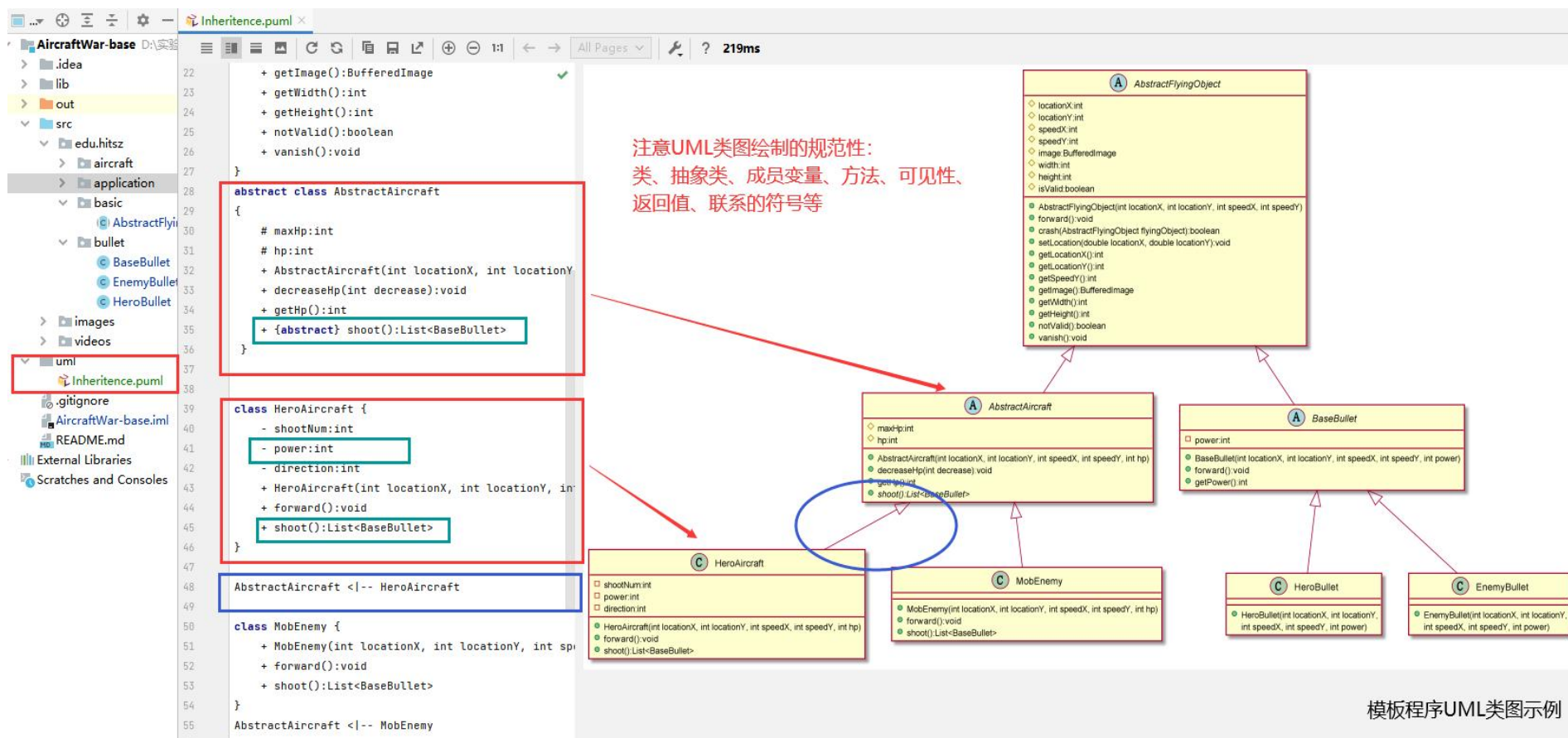
实验步骤：PlantUML插件 (3)

➤ 类与类之间6种关系的画法：



实验步骤：PlantUML插件（4）

➤ 注意：请严格按照指导书4.3.1和4.3.2节要求绘制UML类图。



可通过在箭头内部使用关键字left, right, up 或 down来改变方向。（如：foo -left-> dummyLeft）

实验步骤：英雄机场景分析（1）

英雄机 场景分析

在飞机大战游戏中**只有一种英雄机**，且每局游戏**只有一架英雄机**，由玩家通过鼠标控制其移动。



实验步骤：英雄机场景分析（2）



请思考：

1. 目前代码在哪个类创建英雄机？如何创建？是否符合面向对象设计原则？

```
public Game() {  
    heroAircraft = new HeroAircraft(  
        locationX: Main.WINDOW_WIDTH / 2,  
        locationY: Main.WINDOW_HEIGHT - ImageManager.HERO_IMAGE.getHeight() ,  
        speedX: 0, speedY: 0, hp: 1000);  
}
```

违反
单一职责

X

2. 目前能否保证英雄机的唯一性？

不能，外部程序可以随意用new方法创建一个实例。

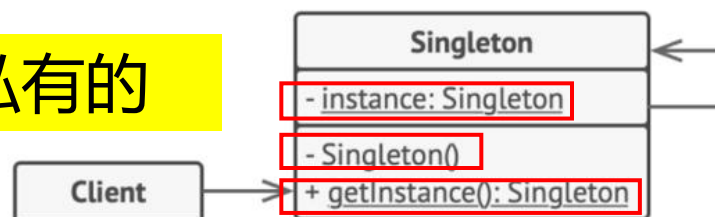
实验步骤：单例模式结构图

单例模式 (Singleton Pattern) 是一种创建型设计模式，能够保证一个类只有一个实例，并提供一个访问该实例的全局节点。

关键代码1：构造函数是私有的

1 单例 (Singleton) 类声明了一个名为 `getInstance` 获取实例 的静态方法来返回其所属类的一个相同实例。

单例的构造函数必须对客户端 (Client) 代码隐藏。调用 获取实例 方法必须是获取单例对象的唯一方式。



关键代码2：提供一个访问该类唯一实例的方法

```
if (instance == null) {
    // 注意：如果程序需要支持多线程，
    // 你必须在此放置线程锁。
    instance = new Singleton()
}
return instance
```

单例模式结构图

实验步骤：单例模式示例代码

① 饿汉式

```
public class EagerSingleton {  
    private static EagerSingleton instance = new EagerSingleton ();  
    private EagerSingleton () {}  
    public static EagerSingleton getInstance() {  
        return instance;  
    }  
}
```

② 懒汉式

```
public class LazySingleton {  
    private static LazySingleton instance = null;  
    private LazySingleton () {}  
    public static synchronized LazySingleton getInstance() {  
        if (instance == null) {  
            instance = new LazySingleton();  
        }  
        return instance;  
    }  
}
```

③ 双重检查锁定 (DCL)

```
public class Singleton {  
    private volatile static Singleton singleton;  
    private Singleton () {}  
    public static Singleton getInstance() {  
        if (singleton == null) {  
            synchronized (Singleton.class) {  
                if (singleton == null) {  
                    singleton = new Singleton();  
                }  
            }  
        }  
        return singleton;  
    }  
}
```



只有敲代码才能
感受到温暖

本次迭代开发目标 (1)

注意：本次迭代仅需完成下列清单任务，实现框架或模拟输出，任务书中其余细节功能将在后续迭代中逐步完善。



任务清单 (CheckList)

UML类图设计

- ① 完善系统继承关系类图、绘制单例模式结构图

设计模式应用

- ② 重构代码，采用单例模式创建英雄机

核心实体类扩展

- ③ 在项目中增加其余4种敌机、5种道具以及它们的抽象父类,完善继承结构
(仅需完成类定义，无需实现复杂逻辑和功能)

敌机生成

- ④ 界面上可观察到普通和精英敌机按周期随机出现

本次迭代开发目标（2）

注意：本次迭代仅需完成下列清单任务，实现框架或模拟输出，任务书中其余细节功能将在后续迭代中逐步完善。



任务清单 (CheckList)

敌机攻击

⑤ 精英敌机按设定周期向下直射单排子弹

道具生成

⑥ 精英敌机坠毁时，以一定概率随机生成一个道具（范围限定为：加血道具、火力道具、超级火力道具）

道具生效

⑦ 英雄机拾取道具后，道具自动生效并消失

道具效果

⑧ 加血道具可使英雄机恢复一定血量，但不超过初始最大值；两种火力道具无需具体实现，生效时只需在控制台打印FireSupply active!类似语句即可。



同学们，请开始实验吧！

THANK YOU